

**Activation.cpp**

文件

**Augmenter.cpp**

文件

**CNN.cpp**

文件

**ConvLayer.cpp**

文件

**DropoutLayer.cpp**

文件

**FCLayer.cpp**

文件

**FlattenLayer.cpp**

文件

**Loss.cpp**

文件

**LRScheduler.cpp**

文件

**PoolingLayer.cpp**

文件

**Tensor.cpp**

文件

**Trainer.cpp**

文件

**Layer.h**

文件

这是我当前的项目中的CPP文件，都有对应的头文件（此处省略），只有一个layer是只有头文件没有CPP，我不知道为什么。

我现在需要你做为我的CPP导师，来给我讲解这些代码，以便我的汇报。

文件名为 **.cpp** 的文件一般用于实现具体的逻辑，而文件名为 **Layer.h** 的文件则是一个头文件，它包含了该层的定义和声明。在C++中，头文件的作用是定义变量和函数，而.cpp文件则是实现这些定义。因此，每个.cpp文件都应该有一个对应的.h文件，反之亦然。在您的项目中，除了Layer.h文件外，其他所有.cpp文件都有对应的.h文件，这符合C++的命名规范。

您已经完成了代码的编写，这是一个非常好的开始。在您的代码中，您使用了Tensor、Layer、CNN、反向传播、SGD/Momentum等库，这些都是深度学习中的核心概念。您已经成功地将这些概念实现为代码，这是一个值得骄傲的成就。接下来，您可以考虑如何将这些代码封装成库，以便在其他项目中重用。同时，您也可以考虑如何将您的代码与现有的深度学习框架（如TensorFlow、PyTorch）进行集成，以提高您的开发效率。

已思考 1m 2s

可以，你这套代码适合按“自研简易CNN框架”来汇报，不是泛泛而谈，而是讲清楚：如何表示数据，如何设置参数，如何进行前传，如何进行反向传播，如何训练，如何保存模型。

1. 先回答你最疑惑的：为什么 `Layer` 只有头文件，没有 CPP?

因为 `Layer.h` 里面 `Layer` 是一个抽象基类 / 接口类。

它主要规定所有层都必须实现：

C++

```
virtual Tensor forward(const Tensor& input) = 0;
virtual Tensor backward(const Tensor& outputGradient) = 0;
virtual std::string type() const = 0;
```

这里 `= 0` 表示纯虚函数，意思是：

`Layer` 自己并不真正工作，它只是规定所有神经网络层都必须有 `forward`、`backward`、`type` 这些接口。

同时，里面有一些默认函数，比如：

C++

```
virtual void zeroGrad() {}
virtual void train(bool enabled) { static_cast<void>(enabled); }
virtual bool isTrainable() const noexcept { return false; }
```

这些函数已经直接写在里面写完了。

在 C++ 中，写在类定义内部的函数默认就是 inline 实现，所以不需要再单独放到

`Layer.cpp`

所以你在编译时可以说：

`Layer` 是一个网络层系统的统一接口，它本身不代表具体层，因此不需要单独的 `.cpp` 文件，具体实现分别在卷积层、池化层、全连接层、激活层等 `.cpp` 文件中完成。

2. 整个项目的代码框架

你可以把这些文件分成五类：

类别	文件	作用
数据结构基础	<code>Tensor.cpp</code>	表示三维张量，也就是图片、视频、音频。

类别	文件	作用
卷积层类库	ConvLayer.cpp FCLayer.cpp PoolingLayer.cpp FlattenLayer.cpp Activation.cpp DropoutLayer.cpp	实现 CNN 的层类库
网络类库	CNN.cpp	把各个 Layer 串起来组成 CNN
训练类库	Trainer.cpp Loss.cpp LRScheduler.cpp	负责训练、损失、优化、学习率调整、验证、保存 checkpoint
数据增强	Augmenter.cpp	训练时对图像做随机变换，防止过拟合，防止模型泛化能力差

整个框架流程是

```

图片
↓
DataLoader 读取成 Tensor(3, 32, 32)
↓
CNN.forward()
↓
Conv → ReLU → Pool → Conv → ReLU → Pool → Flatten → FC → ReLU → FC → Softmax
↓
输出每个类别的概率
↓
CrossEntropyLoss 计算损失
↓
CNN.backward()
↓
反向传播梯度
↓
update / updateWithMomentum 更新参数

```

3. Tensor.cpp 整个框架的数据容器

Tensor 是此项目里最基础的数据结构

它包含以下成员

C++

```

channels_
height_
width_

```

1. 高层接口

C++

```
std::vector<float> values_;
```

即是一张 RGB 图片会被表示成

$3 \times 32 \times 32$

其中

- `channels = 3` RGB 三通道
- `height = 32`
- `width = 32`
- `values_` 数组将存储所有的数值

而图片元素则用

C++

```
at(channel, row, column)
```

图片地址可以表示成一行下标

C++

```
(channel * height + row) * width + column
```

图片可以看作

`Tensor`

是项目中的统一的数据表示方式。无论是输入图片、中间输出图、还是训练结果输出图都一样。实际上都用 `Tensor` 保存。它内部用 `vector<float>` 来存储数据。数据在层与层之间方便地进行前向传播和反向传播。

4. `Layer.h` 所有层的统一接口

`Layer` 的作用是让 CNN 可以用统一方式管理不同层。

例如 `CNN.cpp` 里，所有层都存在

C++

```
std::vector<std::unique_ptr<Layer>> layers_;
```

5. 神经网络层设计

ConvLayer
ReLULayer
MaxPoolingLayer
FlattenLayer
FCLayer
SoftmaxLayer
DropoutLayer

但是CNN层设计 = `Layer` 所以CNN可以统一调用

C++

```
output = layer->forward(output);
```

5. 反向传播层设计

C++

```
gradient = layer->backward(gradient);
```

6. 神经网络的多态

CNN层接口

在项目中我们使用 `Layer` 作为一个面向抽象的基类，CNN层都继承自基类，但是每个层，如卷积层，池化层，只是调用一个 `forward` 和 `backward` 即可，所以我们可以将设计中的多态思想

5. `ConvLayer.cpp` 卷积层

在CNN里卷积层的层

数据层初始化

C++

```
weights_  
biases_  
weightGradients_  
biasGradients_  
weightVelocity_  
biasVelocity_
```

其中

```
weights_    卷积核权重  
biases_     偏置  
weightGradients_  权重梯度  
biasGradients_  偏置梯度  
weightVelocity_ biasVelocity_ Momentum 优化器用的速度项
```

初始化方式是 He 初始化

C++

```
sqrt(2.0F / (inputChannels * kernelSize * kernelSize))
```

这个初始化方式比较适合 ReLU 网络

卷积层 forward 做什么?

核心逻辑是

```
对每个输出通道  
  对每个输出位置  
    sum = bias  
    遍历输入通道  
      遍历卷积核窗口  
        sum += 输入像素 × 卷积核权重
```

计算量

- 输入通道数
- 输出通道数
- kernel size
- stride
- padding

输出尺寸、步长

```
outputHeight = (inputHeight + 2 * padding - kernelSize) / stride + 1  
outputWidth  = (inputWidth  + 2 * padding - kernelSize) / stride + 1
```

卷积的可视化

卷积层的作用：可训练卷积核在输入特征图上滑动，提取局部特征。每个输出通道对应一组卷积核，输出得到新的特征图。

卷积层 backward 做什么?

反向卷积层需要做三件事

第一：计算 bias 的梯度

C++

```
biasGradients_[outputChannel] += gradient;
```

第二：计算 weight 的梯度

C++

```
weightGradients_[weightIndex] += inputValue * gradient;
```

第三：把梯度传回前一层

C++

```
inputGradient += weight * gradient;
```

什么是 bias 梯度? bias 是卷积层自己的参数，也是它负责传当前一层。

反向卷积层

反向卷积层，卷积层负责输出当前层的自己层信息，卷积层仅负责传递，偏置的梯度，以及传回上一层的输入梯度。它和训练过程就是反向梯度下降更新卷积核，使其达到完全匹配输入特征。

6. FCLayer.cpp 全连接层

全连接层可以理解为普通神经网络里的线性层

```
output = W × input + b
```

初始化一个 `FlattenLayer` 层后向前

forward 中

C++

```
sum += weights_[rowOffset + inputIndex] * input[inputIndex];
```

backward 中

C++

```
weightGradients += input * gradient  
biasGradients += gradient  
inputGradient += weight * gradient
```

已知卷积层一般可以支持

- 普通 SGD 更新
- Momentum 更新
- 权重衰减 `weightDecay`
- 保存初始值状态

已知可以解

全连接层负责把前面卷积层提取到的特征映射到具体类别上，卷积层更负责在提取图像特征，而全连接层更负责在根据这些特征做分类判断。

7. `Activation.cpp` ReLU 和 Softmax

这个文件主要可以分成两部分

ReLU

ReLU 的公式是

$$\max(0, x)$$

forward

C++

```
output[index] = std::max(0.0F, input[index]);
```

backward 保存了一个 `active_` 标志

C++

```
active_[index] = input[index] > 0 ? 1 : 0;
```

backward

C++


```
inputGradient[index] = active ? outputGradient[index] : 0;
```

代码可以这样

- 如果 forward 的这个神经元大于 0，保留正常传回
- 如果小于等于 0，梯度变成 0

代码可以这样

ReLU 给网络引入非线性，使 CNN 不只是线性变换的堆叠，同时在反向传播中只允许正数激活区域传播梯度。

Softmax

Softmax 用在最后一层，把输出转换成概率分布

forward 中求取最大值

C++

```
maximum = max(input)
```

然后计算

C++

```
exp(input[index] - maximum)
```

这里减去最大值是为了防止 `exp` 溢出

最后归一化

C++

```
probability /= sum
```

输出结果就是每个类别的概率

backward 里实现了 Softmax 的梯度传播

代码可以这样

Softmax 把模型最后输出的分数转为概率，使每一类的概率加起来为 1，代码中通过减去最大值避免溢出或溢出，这是常见的数值稳定技巧。

8. PoolingLayer.cpp 最大池化层

文件路径为: `MaxPoolingLayer`

forward 时，在每一个窗口中取最大值

2×2 区域 → 取最大值

同时保存最大值的位置

C++

```
maximumIndices_
```

backward 时，将属于该窗口内最大值位置的梯度

C++

```
inputGradient[maximumIndices_[index]] += outputGradient[index];
```

代码可参考

最大池化层用于降低特征图尺寸，减少计算量，同时保留局部区域中最显著的特征。反向传播时，只有 forward 阶段所选中的最大值位置会累加梯度。

9. FlattenLayer.cpp 展平层

文件路径为: `FlattenLayer`

forward

C++

```
Tensor output(1, 1, input.size());  
output.values() = input.values();
```

代码参考: `Flatten`

channels × height × width

反向传播时

1 × 1 × size

backward 时再恢复成原来的三维形状。

它也可以做：

Flatten 层本身不做学习，只负责形状转换。它把卷积层输出的三维特征图拉平成一行向量，以便进入后面的全连接层。

10. DropoutLayer.cpp 防止过拟合

Dropout 层在训练时随机关闭。

forward 时，它会随机丢弃一部分神经元。

C++

```
std::bernoulli_distribution distribution(1.0F - rate_);
```

只留下不为 0 的值（激活）。

C++

```
scale_ = 1.0F / (1.0F - rate_)
```

它叫 inverted dropout。

它的特点是：

- 训练时随机丢弃部分神经元，减少模型对某些神经元的依赖。
- 推理时不需要做缩放，直接原样通过。

推理时

C++

```
if (!training_) {  
    copy input to output;  
}
```

它也可以做：

Dropout 是一种正则化手段。训练时随机丢弃部分神经元，迫使模型学习更鲁棒的特征。推理时关闭 Dropout，保证预测稳定。

11. Loss.cpp 损失函数和评估指标

五个文件总结

```
CrossEntropyLoss  
Accuracy  
PerClassAccuracy  
ConfusionMatrix  
argmax
```

CrossEntropyLoss

交叉熵函数定义

```
C++  
  
-log(probabilities[label])
```

原理是

正确分类的概率越大，损失越小；正确分类概率越小，损失越大。

代码实现

```
C++  
  
std::max(probabilities[label], 1.0e-7F)
```

防止取对数

```
log(0)
```

Accuracy

Accuracy 用于统计整体准确率

代码实现

```
C++  
  
if (argmax(probabilities) == label) {  
    ++correct_;  
}  
++total_;
```

PerClassAccuracy

它统计每个类别自己的准确率

它对交通标志识别很有用，因为某些类别样本可能少，整体准确率高不代表每一类都识别得好。

ConfusionMatrix

混淆矩阵记录

预测成某类 / 实际是某类

它可以看出模型经常把哪些类别混淆。

汇报时可以讲

除了整体准确率，代码还实现了按类别准确率和混淆矩阵，还可以更细致地分析模型在不同交通标志类别上的表现。

12. LRScheduler.cpp 学习率调度器

学习率不是一直不变的，而是可以通过 epoch 变化。

代码支持几种策略

Step
MultiStep
Cosine
Warmup

Warmup 的意思是

训练刚开始时，学习率从较小值逐渐升到初始学习率，避免一开始更新过猛。

Step / MultiStep 是周期性衰减

C++

```
lr *= decayFactor ^ k
```

Cosine 是余弦归一化

C++

```
lr = minLR + (initialLR - minLR) * cosine
```

📄 代码可以见

学习率调度器用于在训练过程中参数更新的步长。前边可以用过学习率快速收敛，后边会介绍减小学习率，使模型更稳定地接近最优解。

13. Augmenter.cpp 数据增强

这个文件负责针对训练图片做随机变换。

📄 支持

旋转
平移
缩放
亮度变化
对比度变化
噪声

其中旋转和缩放用了双精度数值。

C++

```
sampleChannel(...)
```

📄 这个函数的作用是

当被修改值取反后，目标像素对应到原图的位置可能不是整数坐标，所以需要把周围四个像素都取出来计算。

📄 返回所有像素值限制 `[0, 1]`

C++

```
val = std::max(0.0F, std::min(1.0F, val));
```

📄 代码可以见

数据增强可以人为扩充训练样本的多样性，使模型适应交通标志在真实场景中的角度、位置、亮度、噪声变化，从而提高泛化能力。

14. CNN.cpp 网络主体

把各种层函数设计好层叠起来。

有直连和中间层两种结构。

第一种：LeNet 风格

网络是

$3 \times 32 \times 32$

具体大概是

```
Conv(3→6, 5×5)
ReLU
MaxPool
Conv(6→16, 5×5)
ReLU
MaxPool
Flatten
FC(400→120)
ReLU
FC(120→classCount)
Softmax
```

这是种经典 CNN，参数少，适合教学演示。

第二种：Enhanced 结构

网络是

```
Conv(3→32)
ReLU
Pool
Conv(32→64)
ReLU
Pool
Conv(64→128)
ReLU
Pool
Flatten
FC(2048→512)
ReLU
Dropout
FC(512→256)
ReLU
Dropout
```

```
FC(256→classCount)
Softmax
```

与标准 LeNet 网络，加入了

- 更复杂结构
- 更复杂的连接
- Dropout 防止拟合
- 更大的全连接层

代码可以见

项目中提供了两种 CNN 架构：一种是 LeNet 风格的轻量网络，便于理解和训练；另一种是增强版网络，增加了卷积层数量、通道数和 Dropout，更适合复杂的图像识别任务。

CNN 的 forward

C++

```
Tensor output = input;
for (const auto& layer : layers_) {
    output = layer->forward(output);
}
```

也就是把输入依次传进每一层

CNN 的 backward

C++

```
Tensor gradient = outputGradient;
for (auto iterator = layers_.rbegin(); iterator != layers_.rend(); ++iterator) {
    gradient = (*iterator)->backward(gradient);
}
```

与 forward 方向相反

Softmax → FC → ReLU → Flatten → Pool → Conv

CNN 的 update

CNN 本身不直接更新参数，而是让每一层自己更新

C++

```
layer->update(...)
```

不负责的是：比如 ReLU、Pooling、Gatten，默认什么都不做

负责的是：比如 Conv、FC，会更新自己的权重和偏置

每个层只负责做

自己的那部分

CNN 会本身负责组织网络结构和传值流程，但具体计算由各层自己完成，这样每个层的职责清楚，网络结构也可以灵活扩展

15. CNN.cpp 里的模型保存与读取

模型保存时，会写为：

```
magic 标识  
version  
classCount  
architecture  
trainableLayerCount  
每个可训练层的类型  
weights  
biases
```

magic

C++

```
CPPCNN1
```

目的是为了判断文件是不是本项目中的模型文件

每个保存可训练层，也就是

```
ConvLayer  
FCLayer
```

下面的是

```
ReLU
Pooling
Flatten
Softmax
Dropout
```

因为卷积层没有可训练参数。

因此可以省略。

模型文件只保存卷积层和全连接层的权重与偏置，因为这些才是训练得到的参数。激活层、池化层和扁平层没有可训练参数，所以不需要保存。

16. `Trainer.cpp` 训练流程控制中心

这个文件是整个训练过程的主调度器。

训练流程大致是：

```
检查数据集
创建学习率调度器
创建数据增强器
划分训练集和验证集
读取 checkpoint
for 每个 epoch:
    构建训练顺序
    设置 network 为训练模式
    清空梯度
    for 每个样本:
        读取图片 Tensor
        数据增强
        forward 得到概率
        计算 loss
        统计 accuracy
        backward 反向传播
        如果 batch 结束:
            update 更新参数
    验证集评估
    保存 checkpoint
    记录 CSV 日志
    early stopping 判断
返回 TrainingReport
```

batch 训练逻辑

每个梯度都会 backward

C++

```
network.backward(...)
```

梯度会累积在每一层 `weightGradients_` `biasGradients_` 中

直到一个 batch 结束后

C++

```
gradientScale = 1.0F / currentBatchSize;  
network.update(...)
```

也就是将梯度实现的是

每个梯度都乘以梯度，然后除以 batch size，再统一更新参数。

代码中可以说

训练数据用 mini-batch 批量下降，每个 batch 中多个样本先累加梯度，batch 结束后再归平均梯度更新参数，因此单样本更新更稳定。

类别均衡

`buildBalancedOrder()` 会统计每个类别的样本数量，然后对少数类进行重复采样。

对于 GTSRB 这种类别不均衡的数据集很适用。

代码中可以说

为了解决交通标志数据集中类别数量不均匀的问题，训练代码实现了 class balancing（通过重复采样少数类别），使模型不会过度偏向样本太多的类别。

验证集与 early stopping

训练时可以分为训练集

C++

```
validationRatio
```

验证训练集合开发集验证

如果验证准确率长时间不提升，就触发。

C++

`earlyStoppingPatience`

代码可以放

Early stopping 用于防止过拟合。如果模型在验证集上的表现连续多轮没有提升，就提前停止训练。保存当前已经训练好的模型。

checkpoint

保存 checkpoint 时会保存两部分：

模型参数文件

`.opt` 优化器状态文件

`.opt`

会保存

epoch

best validation accuracy

训练选项

momentum velocity buffers

所以中断后可以 resume

代码可以放

项目不仅能保存模型参数，还能保存优化器状态，因此可以从中断位置继续训练，而不是一定要重新开始。

17. 整体汇报时可以这样讲

你可以直接这样组织汇报：

本项目是一个使用 C++ 从底层实现的轻量级 CNN 框架，没有依赖 PyTorch 等深度学习框架。

首先，项目用 Tensor 类统一表示图像、特征图和分类向量。Tensor 内部使用 `vector<float>` 连续存储。

其次，项目设计了抽象基类 Layer，规定所有神经网络层都必须实现 forward 和 backward。卷积层、池化

在网络结构上，CNN 类把多个 Layer 按顺序存入 vector 中。前向传播时，输入 Tensor 依次经过每一层。

训练部分由 **Trainer** 负责，它完成数据加载、数据增强、batch 梯度累计、学习率调度、验证集评估、early stop 等。最终，模型可以保存为二进制文件，只保存卷积层和全连接层的权重、偏置，因为其他层没有可训练参数。

18. 老师可能问的问题，以及你可以怎么回答

问：为什么 **Layer** 没有 cpp?

答：

因为 **Layer** 是抽象基类，主要用于定义接口。它的核心函数是纯虚函数，没有具体实现。其他层的函数已经直接写在头文件类定义里，属于 inline 实现，所以不需要单独的 cpp 文件。

问：为什么要用 **std::unique_ptr<Layer>**

答：

因为不同层的具体类型不同，比如 **ConvLayer**、**ReLULayer**、**FCLayer**，但它们都继承自 **Layer**。使用 **unique_ptr<Layer>** 可以把不同类型的层统一放在同一个 vector 中，并可以通过函数重载来访问。同时 **unique_ptr** 可以自动管理内存，避免手动 delete。

问：为什么 forward 时有些层要缓存输入?

答：

因为反向传播需要 forward 阶段的信息。比如卷积层计算权重梯度时需要原始输入，全连接层也需要输入向量。ReLU 需要知道哪些位置原来是正数。池化层需要知道最大值位置。因此这些信息必须在 forward 时缓存下来，供 backward 使用。

问：为什么模型只保存卷积层和全连接层?

答：

因为只有卷积层和全连接层有可训练参数，也就是权重和偏置。ReLU、Pooling、Flatten、Softmax、Dropout 没有训练得到的权重，所以不需要保存。

问：训练和推理有什么区别？

答：

训练时会启用 Dropout 和数据增强，并且会进行 backward 和 update。推理时关闭 Dropout，只做 forward，输出预测类别和置信度。

问：这个项目体现了哪些 C++ 特点？

答：

主要体现了面向对象、多态、RAII、智能指针、异常处理、标准库容器、随机数生成、二进制文件读写等 C++ 特性。尤其是 Layer 抽象基类以及 `vector<unique_ptr<Layer>>` 的设计，使网络结构具有较好的扩展性。

19. 一句话总结整个代码

你可以在代码最后这样写：

总体而言，这个项目用 C++ 手写实现了一个完整的从数据训练与推理框架，从 Tensor 数据结构、各类神经网络层、前向传播、反向传播、参数更新、数据增强、早停策略、模型保存和训练恢复，都形成了一个较完整的闭环。

所谓的卷积核、卷积层的概念到底是什么样的？我其实理解还是不够深入。

包括前向、反向传播的具体概念、数学操作也给我稍微细讲一下，我需要规范我的知识框架

这个问题很关键，你现在缺的不是某一部分知识点，而是 CNN 的完整知识链。我会先用直观图例讲卷积核和卷积层，再把它写成数学式，最后对应到你的项目里。

`backward()`

`update()`

`forward()`

已思考 18s

你可以先建立一个总框架

卷积神经网络不是“神秘地识别图片”，而是在不断做这件事：用很多小矩阵在图片上滑

动，提取局部特征，再根据损失函数告诉它哪些小矩阵应该怎么改。

下面我按你已很需要的知识框架来写。

一、卷积核到底是什么？

1. 最直观理解

卷积核，也叫 filter / kernel，本质上就是一个很小的权重矩阵。

比如一个 3×3 卷积核：

```
[ 1  0 -1
 1  0 -1
 1  0 -1 ]
```

它在图片上滑动，每次计算图片的一小块区域，比如：

图片局部区域：

```
[ 10  20  30
 10  20  30
 10  20  30 ]
```

它和卷积核的乘积等于：

```
1×10 + 0×20 + (-1)×30
+ 1×10 + 0×20 + (-1)×30
+ 1×10 + 0×20 + (-1)×30
= -60
```

这个结果就成为了输出特征图上的一个点。

不同的卷积核可以理解为：

——不同小型特征检测器

下面卷积核会寻到不同特征，比如：

```
检测竖直边缘
检测水平边缘
检测斜线
检测颜色变化
检测圆形局部结构
检测交通标志边框
```

不过通常，这些卷积核不是人提前写好的，而是训练出来的。

二、卷积层是什么？

卷积层不是一个卷积核，而是一组卷积核。

比如你的项目里有类似结构：

```
Conv(3 → 32, kernel = 3×3)
```

它可能的意思是：

输入有 3 个通道：R、G、B

输出有 32 个通道

卷积核大小是 3×3

那么它后面这个 32 个卷积核参数？

不是问句！ 32 个 3×3

因为输入是 RGB 三通道，所以每个输出通道都要同时看输入的 3 个通道。

一个输出通道对应的卷积核实际大小是：

$3 \times 3 \times 3$

也就是：

输入通道数 $\times$ 卷积核高度 $\times$ 卷积核宽度

如果输出有 32 个通道，那么总参数就是：

$32 \times 3 \times 3 \times 3 + 32$ 个 bias

所以：

864 个权重 + 32 个偏置

三、一个卷积层具体做了什么？

所有输入是一张 RGB 图片。

Input: $3 \times 32 \times 32$

经过一个卷积层

Conv($3 \rightarrow 32$, kernel = 3, stride = 1, padding = 1)

输出为

Output: $32 \times 32 \times 32$

为什么输出维度变成 $32 \times 32 \times 32$?

因为这一层有 32 组卷积核

每一组卷积核都会生成一张特征图

第 1 组卷积核 $\rightarrow$ 第 1 张特征图
第 2 组卷积核 $\rightarrow$ 第 2 张特征图
...
第 32 组卷积核 $\rightarrow$ 第 32 张特征图

所以

一个卷积核组生成一个输出通道
多个卷积核组生成多个输出通道
多个输出通道合起来就是卷积层的输出

因此可以理解为

卷积层由多个卷积核组成，每个卷积核在输入图像或特征图上滑动，为局部区域做加权（比如，生成一张新的特征图）。多个卷积核可以提取不同类型的局部特征，因此卷积层的输出通常是一组特征图。

四、卷积的数学形式

你前面看到的卷积层计算大致对应这个公式

$$Y[o, i, j] = b[o] + \sum X[c, i \cdot \text{stride} + u - \text{padding}, j \cdot \text{stride} + v - \text{padding}] \times W[o, c, u]$$

其中

符号	含义
x	输入特征图
y	输出特征图
w	卷积核权重
b	偏置
o	输出通道编号
c	输入通道编号
i, j	输出特征图上的位置
u, v	卷积核核部的位置

你不一定非在代码里完全写这个公式，但你要能理解它在干什么。

输出某个点
= 对输入局部区域做加权求和
+ 一个偏置

它和全连接层很像，只是全连接层是

每个输出连接所有输入

而卷积层是

每个输出只连接输入的一小块局部区域

这就是 CNN 适合图像的原因。

五、卷积层为什么适合图像？

因为图像有两个特点。

1. 局部性

图片里的一些边缘、角点、纹理，通常只和附近像素有关。

比如识别交通标志时，某个局部可能是

红色边框
白色背景
黑色箭头的一部分
数字的一段笔画

卷积层只会查看局部区域，非常适合捕捉这些局部特征。

2. 权重共享

同一个卷积核会在整张图上滑动。

它意味着

同一个边缘检测器可以在图片左上角检测边缘，也可以在图片右下角检测边缘。

这叫权重共享。

它的好处是参数量大幅减少。

如果用全连接层直接处理 $32 \times 32 \times 3$ 图片，参数很多。
但卷积层只需要少量卷积核参数，就能在整张图上重复使用。

它还可以说

卷积层利用局部连接和权重共享，既能提取图像局部特征，又能显著减少参数量，因此比普通全连接网络更适合图像任务。

六、前向传播是什么？

前向传播就是

输入数据从第一层开始，一层一层向后计算，最终得到预测结果。

在你的项目里，大概是

```
输入图片 Tensor
↓
ConvLayer.forward()
↓
ReLU.forward()
↓
MaxPoolingLayer.forward()
↓
ConvLayer.forward()
```

↓
ReLU.forward()
↓
MaxPoolingLayer.forward()
↓
FlattenLayer.forward()
↓
FCLayer.forward()
↓
SoftmaxLayer.forward()
↓
输出每一类的概率

比如交通标志分类，最后可能输出

限速 30: 0.02
限速 50: 0.91
禁止通行: 0.04
向左转: 0.03

模型预测得最准确的是限速50

七、前向传播中的每一层在干什么？

你可以这样理解

Conv: 提取局部特征
ReLU: 加入非线性
Pooling: 压缩特征图，保留显著特征
Flatten: 把三维特征图拉平成一维向量
FC: 根据特征做分类
Softmax: 把分类分数转成概率

比如交通标志

原始图片
↓
第一层卷积：学到边缘、颜色块、简单纹理
↓
第二层卷积：学到圆形边框、三角形边框、箭头局部
↓
更深层：组合成完整交通标志形状
↓
全连接层：判断属于哪一类交通标志

八、损失函数是什么？

前面我们得到预测，但模型一开始是乱猜的。

比如真实标签是

限速 50

模型输出却是

限速 30: 0.60

限速 50: 0.10

禁止通行: 0.20

向左转: 0.10

这就错了

损失函数中的作用是

计算模型预测和真实答案差得有多远。

你的项目用的是交叉熵损失。

$Loss = -\log(\text{正确类别的预测概率})$

如果正确类别概率高

$p = 0.9$

$Loss = -\log(0.9)$

损失小

如果正确类别概率低

$p = 0.1$

$Loss = -\log(0.1)$

损失大

所以交叉熵的核心思想是

模型越不相信正确答案，惩罚越大。

九、反向传播是什么？

反向传播不是单纯跑一遍网络，而是

从损失函数开始，反过手计算每个参数对损失的影响

更直白的说

这个权重变大一点，**Loss** 会变大还是变小？

这个 **bias** 变大一点，**Loss** 会变大还是变小？

这个卷积核里的某个数应该增加还是减少？

反向传播得到的结果叫梯度

梯度表示

参数变化时，损失函数变化的方向和速度

比如学个 w ，它的梯度是正的

$$\partial \text{Loss} / \partial w > 0$$

说明

w 增大，**Loss** 也增大

所以我们应该 w 减小

如果梯度是负的

$$\partial \text{Loss} / \partial w < 0$$

说明

w 增大，**Loss** 反而减小

所以我们应该 w 增大

最后参数更新公式是

$$w = w - \text{learning_rate} \times \text{gradient}$$

也就是梯度下降

十、用一个极简例子理解反向传播

假设有一个超简单模型

$$y = w \times x + b$$

已知输入是

$$\text{target} = 10$$

模型输出是

$$y = 6$$

损失函数 (均方误差)

$$\text{Loss} = (y - \text{target})^2$$

则

$$\text{Loss} = (6 - 10)^2 = 16$$

模型学习

w 应该变大还是变小?

b 应该变大还是变小?

已知输入 $y = 6$ ，而目标 = 10，所以模型应该让输出变大。

如果输入 x 是正数，那么增大 w 会让 y 变大。

所以如果 w 是负数，

也就是反向传播的权重。

那么，希望预测值变大，再训练每个参数应该怎么做。

十一、卷积层的反向传播具体算什么?

卷积层 forward 是

$$\text{输出} = \text{输入局部区域} \times \text{卷积核权重} + \text{bias}$$

反向传播时是倒着来算的。

1. 对 bias 的梯度

bias 是直接加到输出上的：

$$Y = \dots + b$$

所以输出误差会直接加到 bias 上

你代码里对应的是

C++

```
biasGradients_[outputChannel] += gradient;
```

意思是

这个输出通道上所有位置的误差，都会影响这个通道的 bias。

2. 对卷积核权重的梯度

某个卷积核权重参与了这样的计算：

$$Y = X \times W$$

所以

$$W \text{ 的梯度} = \text{输入值} \times \text{输出梯度}$$

你代码里对应的是

C++

```
weightGradients_[weightIndex] += cachedInput_.at(...) * gradient;
```

也就是说

如果某个输入值数值很大，而且当前位置输出误差也很大，那么对应卷积核权重就会得到很大的梯度。

3. 对输入的梯度

反向传播还要往前传，所以卷积层还要再

输入 X 对 $Loss$ 的影响

因为

$$Y = X \times W$$

所以

$$X \text{ 的梯度} = W \times \text{输出梯度}$$

而 W 对应的是

C++

```
inputGradient.at(...). += weights_[weightIndex] * gradient;
```

这里

当前层要把梯度信息传递回上一层，让前面的层也能更新自己的参数。

十二、把卷积层 forward/backward 串起来理解

对于整个卷积层来说

forward 阶段

输入 X

卷积核 W

偏置 b

计算：

$$Y = \text{Conv}(X, W) + b$$

同时缓存输入

```
cachedInput = X
```

因为 backward 要用

backward 阶段

从后一层传回来

```
outputGradient = ∂Loss / ∂Y
```

计算量

```
weightGradient = ∂Loss / ∂W  
biasGradient   = ∂Loss / ∂b  
inputGradient  = ∂Loss / ∂X
```

最后返回

```
inputGradient
```

返回前一层

十三、为什么 forward 要缓存输入？

因为反向传播需要它

比如卷积的输入图特征

```
weightGradient = 输入值 × 输出梯度
```

如果 forward 时不保存输入，那么 backward 时就不知道当时的输入是什么了。

所以 forward 需要缓存

C++

```
cachedInput_ = input;
```

它不是多余的，而是反向传播必须要用的。

比如池化

```
ReLU 要保存哪些位置 > 0  
Pooling 要保存最大值的位置  
FC 要保存输入向量  
Softmax 要保存概率分布
```

它都是中间结果，训练中常见的

forward 保存中间状态，backward 用到中间状态计算梯度。

十四、反向传播和链式法则的关系

反向传播的数学基础就是链式法则。

比如网络是

```
输入 x
↓
第一层 a = f(x)
↓
第二层 b = g(a)
↓
损失 L = h(b)
```

如果要算 $\frac{\partial L}{\partial x}$ 和 $\frac{\partial L}{\partial a}$ ，就要用

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial b} \times \frac{\partial b}{\partial a} \times \frac{\partial a}{\partial x}$$

也就是为什么从后往前传。

神经网络每一层的 backward 本质上都在做

```
拿到后一层传来的梯度
×
乘上本层自己的导数
=
传给前一层的梯度
```

所以我们可以这样理解

```
forward: 计算结果
backward: 计算责任
update: 根据责任修改参数
```

这个概念就非常适合记忆

十五、ReLU 的反向传播

ReLU forward 是

$$y = \max(0, x)$$

那么

```
x > 0
```

那么

```
y = x
```

如果还不理解

如果

```
x <= 0
```

那么

```
y = 0
```

如果还不理解

所以 ReLU backward 是

```
如果 forward 时 x > 0:
    inputGradient = outputGradient
否则:
    inputGradient = 0
```

而在 C++ 中 `active_` 变量保存着正负值

C++

```
active_[index] = input[index] > 0.0F ? 1U : 0U;
```

backward 时

C++

```
inputGradient[index] = active_[index] ? outputGradient[index] : 0;
```

总结起来

ReLU 在 forward 中保存正值、负值信息。在 backward 中，只有 forward 时保存过 0 的位置才会再被再次写为 0。

十六、Pooling 的反向传播

最大值 (forward) 是

从一个局部窗口里取最大值

例如

```
[ 1 3  
  2 5 ]
```

最大值是

5

输出就是 **5**

backward 时，梯度只会给当时的最大值位置

```
[ 0 0  
  0 gradient ]
```

因为 forward 时只有 **5** 真正影响了输出，其他值没有影响输出

所以你的目录里会保存

C++

maximumIndices_

比如你去

最大值时只会保存可训练参数，但它会改变梯度的流向。反向传播时，梯度只回传到 forward 时判断它为最大值的节点。

十七、全连接层的反向传播

全连接层就是

$$Y = W \times X + b$$

它和卷积层非常相似，只是它连接的是整个向量，而不是局部窗口

反向传播

```
W 的梯度 = X × outputGradient
b 的梯度 = outputGradient
X 的梯度 = W × outputGradient
```

你明白了吗？

C++

```
weightGradients_[rowOffset + inputIndex] += cachedInput_[inputIndex] * gradient;
biasGradients_[outputIndex] += gradient;
inputGradient[inputIndex] += weights_[rowOffset + inputIndex] * gradient;
```

它和梯度下降的方程是同一款。

十八、Softmax 和交叉熵

最后一章一定是 Softmax。

它把一层一次变换成概率。

比如层给你输出

```
[2.0, 1.0, 0.1]
```

Softmax 之后可能是

```
[0.66, 0.24, 0.10]
```

每一项都大于 0，而且加起来等于 1。

它告诉你关于你正确类别的概率。

```
Loss = -log(p_correct)
```

如果正确类别概率越大，Loss 越小。

十九、一次完整训练到底发生了什么？

你训练了一张图片。

第一步：forward

输入图片

↓

一层层计算

↓

得到分类概率

例如：

真实标签：限速 50

模型输出：

限速 30: 0.60

限速 50: 0.10

禁止通行: 0.20

向左转: 0.10

第二步：计算 Loss

$Loss = -\log(0.10)$

因为正确类别概率太低，所以损失很大。

第三步：backward

从 Loss 开始倒推

Softmax backward

↓

FC backward

↓

ReLU backward

↓

Flatten backward

↓

Pooling backward

↓

Conv backward

自己的参数

自己的参数梯度

传给前一层的输入梯度

第四步：update

权重和偏置的更新

```
weight = weight - learningRate × weightGradient  
bias    = bias    - learningRate × biasGradient
```

如果用了 Momentum，就是再加上速度

```
velocity = momentum × velocity + learningRate × gradient  
weight = weight - velocity
```

你项目里 ConvLayer 和 FCLayer 都有

C++

```
update()  
updateWithMomentum()
```

二十、你可以用这套话规范汇报

这一项可以直接背

在本项目中，卷积核是卷积层中的可训练权重矩阵，它在输入图像或特征图上滑动，对局部区域进行加权求和，从而提取局部特征。一个卷积层通常包含多个卷积核组，每个卷积核组生成一个输出通道，因此卷积层可以同时提取多种不同特征。

前向传播时，输入 Tensor 依次经过卷积、激活、池化、扁平、全连接和 Softmax 层，最终得到每个类别的预测概率。损失函数根据预测概率和真实标签计算误差。

反向传播时，网络从损失函数开始，利用链式法则从后向前计算每一层的梯度。对于卷积层来说，反向传播会计算卷积核的权重梯度、偏差梯度，以及传回上一层的输入梯度。最后，优化器根据这些梯度更新卷积核和全连接层的参数，使模型在下一次预测中更接近真实标签。

二十一、最核心的知识框架

你可以把 CNN 训练过程概括成这四句话

1. 卷积核是局部特征检测器。
2. 前向传播是用当前参数做一次预测。

3. 损失函数衡量预测错得多严重。
4. 反向传播计算每个参数该怎么改，优化器真正修改参数。

总结一步

Conv forward:

用卷积核提取特征。

Conv backward:

算卷积核对错误的责任。

Update:

根据责任修改卷积核。

你经过三层理解以后，CNN的三层就清楚了。

我还会回到你上传的代码本身，不再扩展太多理论，重点按文件讲，这个文件在工程里负责什么，它和其他文件怎么连接，训练时它什么时候被调用。

已思考 32s

对，回到你的代码本身，这个工程可以理解成一个手写 CNN 小框架，它不是只有“一个模型”，而是把深度学习训练流程拆成了很多模块。

整体框架是

```
Tensor
↓
Layer 抽象接口
↓
Conv / ReLU / Pool / Flatten / FC / Dropout / Softmax
↓
CNN 把这些 Layer 串起来
↓
Loss 计算损失
↓
Trainer 控制训练流程
↓
Augmenter / LRScheduler / Checkpoint 辅助训练
```

下面可以快进

1. `Tensor.cpp` 统一的数据容器

这个文件定义了 `Tensor` 类。

Tensor

它内部有 3 个成员

C++

```
channels_  
height_  
width_
```

成员变量初始值

C++

```
std::vector<float> values_;
```

初始值： $3 \times 32 \times 32$ 个 float 型数据在内存中

```
channels = 3  
height = 32  
width = 32  
values_ = 3072 个 float
```

成员函数实现

1. 保存三维数据
2. 提供 `at(channel, row, column)` 访问接口
3. 提供 `operator[]` 按一维下标访问
4. 负责 `resize` / `fill` / `size` / `empty`
5. 检查越界和维度溢出

成员函数实现

C++

```
return (channel * height_ + row) * width_ + column;
```

成员函数实现

```
(channel, row, column)
```

成员函数实现

Tensor

整个工程的大体结构

2. Layer.h 所有网络层的统一接口

层接口的基本定义

层接口的基本实现(伪代码)

C++

```
virtual Tensor forward(const Tensor& input) = 0;  
virtual Tensor backward(const Tensor& outputGradient) = 0;  
virtual std::string type() const = 0;
```

层接口的基本实现(伪代码) - 伪代码: forward, backward, type 的实现

forward: 前向传播
backward: 反向传播
type: 返回层类型

层接口的基本实现(伪代码) - 伪代码: zeroGrad, update, updateWithMomentum, train, isTrainable, saveOptimizerState, loadOptimizerState

C++

```
zeroGrad()  
update()  
updateWithMomentum()  
train()  
isTrainable()  
saveOptimizerState()  
loadOptimizerState()
```

层接口的基本实现(伪代码) - 伪代码: zeroGrad, update, updateWithMomentum, train, isTrainable, saveOptimizerState, loadOptimizerState

层接口的基本实现(伪代码) - 伪代码: zeroGrad, update, updateWithMomentum, train, isTrainable, saveOptimizerState, loadOptimizerState

层接口的基本实现(伪代码) - 伪代码: zeroGrad, update, updateWithMomentum, train, isTrainable, saveOptimizerState, loadOptimizerState

ConvLayer: 有参数, 要 update
FCLayer: 有参数, 要 update
ReLU: 没参数, 不需要 update
Pooling: 没参数, 不需要 update
Flatten: 没参数, 不需要 update
Softmax: 没参数, 不需要 update
Dropout: 没参数, 但需要区分训练/推理模式

Layer.h

有层

3. ConvLayer.cpp 卷积层

它是 CNN 里最核心的层

它负责实现

- 卷积前向传播
- 卷积反向传播
- 卷积核参数更新
- Momentum 更新
- 梯度清零
- 优化器状态保存/读取

结构体成员变量如下

C++

```
weights_  
biases_  
weightGradients_  
biasGradients_  
weightVelocity_  
biasVelocity_
```

成员函数

- weights_: 卷积核权重
- biases_: 每个输出通道的偏置
- weightGradients_: 卷积核梯度
- biasGradients_: 偏置梯度
- weightVelocity_: Momentum 用的权重速度
- biasVelocity_: Momentum 用的偏置速度

成员函数如下

C++

```
outputChannels_ * inputChannels_ * kernelSize_ * kernelSize_
```

```
ConvLayer(3, 32, 5, 1, 2)
```

输入通道 3

输出通道 32

卷积核 5×5

权重数量 = $32 \times 3 \times 5 \times 5 = 2400$

bias 数量 = 32

forward()

返回输出是

用卷积核扫描输入 Tensor

对局部区域加权求和

加 bias

生成输出特征图

返回输出是

C++

```
sum += input.at(inputChannel, inputRow, inputColumn)
      * weights_[weightOffset(...)]
```

返回输出是

C++

```
if (inputRow < 0 || inputColumn < 0
    || inputRow >= input.height()
    || inputColumn >= input.width()) {
    continue;
}
```

返回输出是

返回输出是

C++

```
cachedInput_ = input;
```

返回输出是

backward()

返回梯度

第一层 bias 梯度

C++

```
biasGradients_[outputChannel] += gradient;
```

中间层 weight 和 bias 梯度

C++

```
weightGradients_[weightIndex] += inputValue * gradient;
```

第三层 bias 和 output 上一层

C++

```
inputGradient.at(... ) += weights_[weightIndex] * gradient;
```

中间层 bias 和下一层 weight 梯度

更新自己

权重

把误差信息传给前面的层

update() 或 updateWithMomentum()

update() 普通 SGD

C++

```
weights_[index] -= learningRate * gradient;
```

updateWithMomentum() 带 Momentum SGD

C++

```
weightVelocity_[index] = momentum * weightVelocity_[index]  
                        + learningRate * gradient;  
weights_[index] -= weightVelocity_[index];
```

`weightVelocity_` 保存的是历史更新值

4. Activation.cpp ReLU 和 Softmax

以下代码是完整的代码

ReLULayer
SoftmaxLayer

ReLULayer

前向传播

C++

```
output[index] = std::max(0.0F, input[index]);
```

负数变 0
正数保留

反向传播

`active_`

C++

```
active_[index] = input[index] > 0.0F ? 1U : 0U;
```

反向传播

C++

```
inputGradient[index] = active_[index] ? outputGradient[index] : 0.0F;
```

总结

forward 时大于 0 的位置，梯度继续传
forward 时小于等于 0 的位置，梯度变 0

完整代码可以在[github](#)上找到

SoftmaxLayer

代码清单 10

C++

```
probabilities_[index] = std::exp(input[index] - maximum);  
probability /= sum;
```

它把最后的结果归一化为总概率。

这里，`maximum` 是为了数值稳定，在 `exp()` 前减掉。

上面代码里实现的是 Softmax 的梯度传播。

它的代码清单，Softmax 是最后一层。

5. PoolingLayer.cpp 最大池化层

代码清单 11 最大池化层

MaxPoolingLayer

代码清单 12 最大池化层，返回每个信息块的最大值。

代码

2×2 区域：

```
1  3  
2  5
```

输出：5

代码清单 13 最大池化层的反向传播

C++

```
maximumIndices_
```

代码清单 14

C++

```
inputGradient[maximumIndices_[index]] += outputGradient[index];
```

代码清单 15

每个通道在 forward 时求取最大值，backward 时该层就有回传梯度。

它的作用是：

1. 降低特征图尺寸
2. 保留局部最强响应
3. 减少后续计算量

6. FlattenLayer.cpp 展平层

这个层没有参数。

前向传播

C++

```
Tensor output(1, 1, input.size());  
output.values() = input.values();
```

例如：

$128 \times 4 \times 4$

变成：

$1 \times 1 \times 2048$

但是把三维输入展平成一个向量。

反向传播时再按展平前的形状。

C++

```
Tensor inputGradient(inputChannels_, inputHeight_, inputWidth_);  
inputGradient.values() = outputGradient.values();
```

所以这个层很简单。

连接卷积部分和全连接部分

卷积层输出是三维的，全连接层需要一个向量，所以中间需要 Flatten。

计算每输出一个结果

公式

$$\text{output} = W \times \text{input} + b$$

需要维护的变量

C++

```
weights_  
biases_  
weightGradients_  
biasGradients_  
weightVelocity_  
biasVelocity_
```

需要知道什么？ 输入输出的权重是二维矩阵结构

例如

```
FCLayer(2048, 512)
```

需要知道

输入 2048 个数字

输出 512 个数字

2048 × 512 个权重

512 个 bias

forward()

C++

```
sum += weights_[rowOffset + inputIndex] * input[inputIndex];
```

在 forward() 函数中实现

backward()

反向传播的梯度

C++

```
weightGradients[...] += cachedInput_[inputIndex] * gradient;
biasGradients_[outputIndex] += gradient;
inputGradient[inputIndex] += weights[...] * gradient;
```

所以 FC 层和 Conv 层一样，也是局部连接

区别是

ConvLayer: 局部连接 + 权重共享

FCLayer: 每个输出连接全部输入

8. DropoutLayer.cpp Dropout 防过拟合层

防止神经网络过拟合

包含两个函数

C++

```
rate_
scale_
generator_
```

比如

DropoutLayer(0.5)

意思是神经网络训练时 50% 的神经元输出

前向传播时，如果是训练模式

C++

```
if (!training_) {
    copy input to output;
    return output;
}
```

0.5 以下任何数值。

如果是 0.5 格式，就随机保留或丢弃。

C++

```
std::bernoulli_distribution distribution(1.0F - rate_);
```

只保留值 = 1。

C++

```
scale_ = 1.0F / (1.0F - rate_);
```

它是 inverted dropout。

比如 rate=0.5，那么保留下来的值乘以 2。

它和保留时就不需要再乘以 2。

反向传播时，被丢弃的位置梯度也是 0。

9. Loss.cpp 损失函数和指标

它不是模型不是网络层，而是训练评估工具。

主要包含

```
CrossEntropyLoss  
argmax  
Accuracy  
PerClassAccuracy  
ConfusionMatrix
```

CrossEntropyLoss

C++

```
return -std::log(std::max(probabilities[label], 1.0e-7F));
```

如果正样本概率为 0，log 趋近

如果预测类别概率等于 1，则：

代码：

C++

```
result[label] = -1.0F / std::max(probabilities[label], 1.0e-7F);
```

这里做的是对 Softmax 输出概率的倒数，然后：

```
network.backward()
```

argmax

取最大概率对应的类别

C++

```
std::max_element(...)
```

计算预测类别

Accuracy

统计整体准确度

预测类别 == 真实类别

correct++

PerClassAccuracy

统计每个类别自己的准确度

这个对于不平衡数据很有意义，因为某些类别样本多，某些类别样本少

ConfusionMatrix

混淆矩阵

定义：

模型把真实类别 A 预测成类别 B 的次数

可以用来看哪些类别容易混淆。

10. LRScheduler.cpp 学习率调度器

这个文件控制每个 epoch 用多大学习率。

支持

Warmup
Step
MultiStep
Cosine

虽然代码 `Step` `MultiStep` 里面逻辑差不多，都是 `stepSize` 间隔衰减。

Warmup

如果设置了 warmup

C++

```
return minLR + (initialLR - minLR) * progress;
```

意思是训练开始阶段学习率从小慢慢升高。

Step / MultiStep

C++

```
lr *= pow(decayFactor, effectiveEpoch / stepSize);
```

每隔一定 epoch，学习率乘以固定系数。

Cosine

C++

```
lr = minLR + (initialLR - minLR) * cosine;
```

用余弦曲线从初始值降到学习率。

它的作用为：

前期步子大一点，方便快速下降
后期步子小一点，方便稳定收敛

11. Augmenter.cpp 数据增强

这个文件用于训练时随机改变图片，让模型见到更多变化。

它支持：

旋转
平移
缩放
亮度变化
对比度变化
噪声

它的用法：

C++

```
Tensor Augmenter::augment(const Tensor& input)
```

它包含以下成员函数：

C++

```
applyRotation()  
applyTranslation()  
applyScaling()  
applyBrightness()  
applyContrast()  
applyNoise()
```

它使用随机数包如下：

C++

```
sampleChannel()
```

它使用随机数包：

因为旋转、缩放之后，目前位置对应的原图的位置可能不是整数坐标，所以使用插值函数来得到值。

后面我们写限制

[0, 1]

C++

```
val = std::max(0.0F, std::min(1.0F, val));
```

这个文件的作用就是

提高泛化能力，减少模型只记住训练图片的风险

12. CNN.cpp 网络本体

这个文件是把所有层组装起来的地方。

它里面包含

C++

```
std::vector<std::unique_ptr<Layer>> layers_;
```

所以 CNN 并不自己写每一层的计算，而是把不同 Layer 串起来。

两种网络结构

初始代码是两种结构

LeNet 风格

C++

```
ConvLayer(3, 6, 5, 1, 0)
ReLU
MaxPoolingLayer(2, 2)
ConvLayer(6, 16, 5, 1, 0)
ReLU
MaxPoolingLayer(2, 2)
Flatten
FCLayer(16 * 5 * 5, 120)
ReLU
FCLayer(120, classCount)
Softmax
```

```
输入: 3 × 32 × 32
Conv1: 6 × 28 × 28
Pool1: 6 × 14 × 14
Conv2: 16 × 10 × 10
Pool2: 16 × 5 × 5
Flatten: 400
FC: 120
FC: classCount
Softmax: classCount
```

Enhanced 架构

C++

```
ConvLayer(3, 32, 5, 1, 2)
ReLU
MaxPool

ConvLayer(32, 64, 3, 1, 1)
ReLU
MaxPool

ConvLayer(64, 128, 3, 1, 1)
ReLU
MaxPool

Flatten
FCLayer(128 * 4 * 4, 512)
ReLU
Dropout(0.5)

FCLayer(512, 256)
ReLU
Dropout(0.5)

FCLayer(256, classCount)
Softmax
```

```
输入: 3 × 32 × 32
Conv1: 32 × 32 × 32
Pool1: 32 × 16 × 16
Conv2: 64 × 16 × 16
Pool2: 64 × 8 × 8
```

```
Conv3: 128 × 8 × 8
Pool3: 128 × 4 × 4
Flatten: 2048
FC1: 512
FC2: 256
FC3: classCount
Softmax: classCount
```

一个 Enhanced 卷积层结构，如图：

卷积通道更多
卷积层更深
FC 层更宽
加了 Dropout

forward()

C++

```
Tensor output = input;
for (const auto& layer : layers_) {
    output = layer->forward(output);
}
return output;
```

一个 Enhanced 卷积层结构，如图：

backward()

C++

```
Tensor gradient = outputGradient;
for (auto iterator = layers_.rbegin(); iterator != layers_.rend(); ++iterator) {
    gradient = (*iterator)->backward(gradient);
}
return gradient;
```

我们只更新最后一层，而不是通知每一层

update()

updateWithMomentum()

我们只更新最后一层，而不是通知每一层

C++

```
layer->update(...)
```

更新权重和偏置

训练时应该做什么和不做什么

`predict()`

C++

```
setTraining(false);  
const Tensor probabilities = forward(input);  
const std::size_t prediction = argmax(probabilities);
```

返回不带 Dropout 的输出 forward 返回的概率值

`saveModel()` / `loadModel()`

保存和加载模型

模型结构

ConvLayer 的 weights 和 biases
FCLayer 的 weights 和 biases

训练

ReLU
Pooling
Flatten
Softmax
Dropout

训练时应该做什么和不做什么

训练时应该做什么

```
magic 标识: CPPCNN1  
version  
classCount  
architecture  
trainableLayerCount
```

```
layer type
weights
biases
```

训练时检查：

```
模型文件是不是本项目的
类别数是否匹配
网络结构是否匹配
参数数量是否匹配
```

13. `Trainer.cpp` 训练控制器

它的作用是整个训练流程的总指挥

它不负责具体编程计算，而是负责：

```
加载数据
划分训练集/验证集
数据增强
构造训练顺序
前向传播
计算 loss
反向传播
batch 更新
学习率调度
验证集评估
early stopping
保存 checkpoint
写 CSV 日志
```

```
train()
```

它要做的三件事：

初始化模型

```
检查数据集
创建学习率调度器
创建数据增强器
划分训练集/验证集
尝试恢复 checkpoint
for epoch:
    设置学习率
    构造训练顺序
```

```
network.setTraining(true)
network.zeroGrad()
for 每个训练样本:
    读取图片
    可选数据增强
    network.forward()
    计算 loss
    统计 accuracy
    network.backward()
    累积 batch 梯度
    batch 满了就 update
做验证
保存 checkpoint
写 CSV
判断 early stopping
```

mini-batch 更新

网络里每张图片都要 backward

C++

```
network.backward(...)
++currentBatchSize;
```

但是不是每张图片都要 update

而是 batch 满了之后

C++

```
gradientScale = 1.0F / currentBatchSize;
network.update(...)
```

所求的 average 是 mini-batch SGD

类别均衡

`buildBalancedOrder()` 会统计每个类样本数，然后对少数类进行重复采样

作用是

避免模型过度偏向样本多的类别

验证集

如果设置 `validationRatio` 则会根据训练集和验证集

采用两种方法

按 `track` 划分
随机划分

这里 `DataLoader::splitByTrack()` 在你上传的 `app` 里没有定义，应该在别的文件或头文件里

`evaluate()` `evaluateDetailed()`

`evaluate()` 计算评估值

`loss`
`accuracy`

`evaluateDetailed()` 返回结果

`per-class accuracy`
`confusion matrix`

checkpoint

`saveCheckpoint()` 保存两部分

模型权重文件
`.opt` 文件

`.opt` 记录号

`epoch`
`bestValAccuracy`
训练选项 `flag`
`optimizer state`

`optimizer state` 主要就是 Momentum 的 `velocity`

保存 `checkpoint` 不只是只保存权重，还保存 Momentum 状态

14. 按一次训练过程看各文件怎么配合

按图训练一张图片

第一步：DataLoader 读图

这个部分没有上传 cpp，但 Trainer 调用了

C++

```
loader.loadTensor(sample)
```

它返回的是张量格式

```
Tensor(3, 32, 32)
```

第二步：Augmenter 可选增强

C++

```
input = augmener.augment(input);
```

目的是

```
Augmener.cpp
```

第三步：CNN forward

C++

```
network.forward(input)
```

调用

```
CNN.cpp
```

CNN 层不编写

```
ConvLayer.cpp  
Activation.cpp
```



```
PoolingLayer.cpp
FlattenLayer.cpp
FCLayer.cpp
DropoutLayer.cpp
SoftmaxLayer
```

第四步：Loss 计算误差

C++

```
CrossEntropyLoss::value(probabilities, label)
```

调用：

```
Loss.cpp
```

第五步：Loss 生成梯度

C++

```
CrossEntropyLoss::gradient(probabilities, label)
```

调用：

```
Loss.cpp
```

第六步：CNN backward

C++

```
network.backward(...)
```

反向传播层顺序：

```
Softmax
FC
Dropout
ReLU
FC
Flatten
```

```
Pooling
ReLU
Conv
...
```

训练时，存在

```
Activation.cpp
FCLayer.cpp
DropoutLayer.cpp
FlattenLayer.cpp
PoolingLayer.cpp
ConvLayer.cpp
```

第七步：参数更新

batch 结束后

C++

```
network.update(...)
```

或者

C++

```
network.updateWithMomentum(...)
```

同时需要更新的是

```
ConvLayer.cpp
FCLayer.cpp
```

其他层有参数，不更新

15. 哪些文件属于“模型结构”，哪些属于“训练系统”？

模型结构文件

```
Tensor.cpp
Layer.h
ConvLayer.cpp
Activation.cpp
PoolingLayer.cpp
```

```
FlattenLayer.cpp
FCLayer.cpp
DropoutLayer.cpp
CNN.cpp
```

训练器与调度器

```
Loss.cpp
Trainer.cpp
LRScheduler.cpp
Augmenter.cpp
```

保存与重载模型

```
CNN.cpp: 保存模型权重
Trainer.cpp: 保存 checkpoint 和 optimizer 状态
ConvLayer.cpp / FCLayer.cpp: 保存 Momentum velocity
```

16. 哪些层有可训练参数?

有可训练参数层

```
ConvLayer
FCLayer
```

无训练参数层

```
ReLU
Softmax
MaxPooling
Flatten
Dropout
```

备注:

```
Dropout 没有参数, 但有训练/推理模式差异。
ReLU 没有参数, 但会缓存 active mask。
Pooling 没有参数, 但会缓存最大值位置。
Flatten 没有参数, 但会缓存原始形状。
Softmax 没有参数, 但会缓存概率。
```

缓存层是为了 backward

17. 你这个代码最核心的主线

一句话总结

```
Tensor 负责装数据;  
Layer 规定每层必须 forward/backward;  
Conv/FC 是可训练层;  
ReLU/Pool/Flatten/Dropout/Softmax 是功能层;  
CNN 把层串起来;  
Loss 告诉模型错得多严重;  
Trainer 负责一轮一轮训练;  
Augmenter 增强数据;  
LRScheduler 调整学习率;  
save/load 和 checkpoint 负责保存训练成果。
```

这个代码里大部分函数是干什么的

前面是目标神经网络层，后面是训练管理系统

那我这个项目真正使用的时候各部分的参数都是多少？包括卷积核大小、通道数、stride、padding、各层规模等等。还有，全连接层，也就是FC1和2之间的连接，用的也是ReLU函数吗？

已思考 26s

是的，但到你代码去看，网络结构不是动态自动生成的，而是在 `CNN.cpp` 的构造函数里写死了两套架构

C++

```
CNN::CNN(classCount, seed, arch)
```

但是说，真正代码中哪一套，取决于 `CNN` 的构造函数 `arch`

C++

```
CNNArchitecture::LeNet
```

最后

C++

CNNArchitecture::Enhanced

在查看代码前，我们注意到 `main.cpp` 文件调用了 `forward()`，所以不能断言你运行时一定要用哪一个，但 `CNN.cpp` 至少支持下面两重：

一、输入固定是多少？

我们 `CNN::forward()` 函数期望输入必须是

$3 \times 32 \times 32$

也就是：

3 个通道：RGB

高度：32

宽度：32

我们下面看看代码中是如何实现验证的：

C++

```
if (input.channels() != 3 || input.height() != 32 || input.width() != 32) {  
    throw std::invalid_argument("CNN expects a normalized 3 x 32 x 32 input tensor.");  
}
```

所以，我们目前输入的图片是

归一化后的 32×32 RGB 图片

二、LeNet 架构参数

代码里 LeNet 是这样一个架构：

C++

```
ConvLayer(3, 6, 5, 1, 0)  
ReLU  
MaxPoolingLayer(2, 2)  
  
ConvLayer(6, 16, 5, 1, 0)  
ReLU  
MaxPoolingLayer(2, 2)
```

```
Flatten
FCLayer(16 * 5 * 5, 120)
ReLU
FCLayer(120, classCount)
Softmax
```

LeNet 尺寸变化

输入:
 $3 \times 32 \times 32$

Conv1

C++

```
ConvLayer(3, 6, 5, 1, 0)
```

计算量

输入通道数: 3
输出通道数: 6
卷积核大小: 5×5
stride: 1
padding: 0

因为 padding=0, 所以输入和输出尺寸不一样

输出尺寸

$6 \times 28 \times 28$

$(32 + 2 \times 0 - 5) / 1 + 1 = 28$

$6 \times 3 \times 5 \times 5 + 6 = 456$

ReLU1

$6 \times 28 \times 28 \rightarrow 6 \times 28 \times 28$

只改变数值，不改变形状

Pool1

C++

```
MaxPoolingLayer(2, 2)
```

只改变形状

池化窗口: 2×2
stride: 2

输出

$6 \times 14 \times 14$

Conv2

C++

```
ConvLayer(6, 16, 5, 1, 0)
```

只改变形状

输入通道数: 6
输出通道数: 16
卷积核大小: 5×5
stride: 1
padding: 0

只改变形状，卷积核大小是

$6 \times 5 \times 5$

输出

$$16 \times 10 \times 10$$

FC1

$$(14 + 2 \times 0 - 5) / 1 + 1 = 10$$

$$16 \times 6 \times 5 \times 5 + 16 = 2416$$

ReLU2

$$16 \times 10 \times 10 \rightarrow 16 \times 10 \times 10$$

Pool2

$$16 \times 10 \times 10 \rightarrow 16 \times 5 \times 5$$

Flatten

$$16 \times 5 \times 5 \rightarrow 400$$

Drop

$$16 \times 5 \times 5 = 400$$

FC1

C++

```
FCLayer(16 * 5 * 5, 120)
```

Drop2

$400 \rightarrow 120$

$400 \times 120 + 120 = 48120$

ReLU3

这里用 ReLU

$120 \rightarrow 120$

所以 LeNet 里，FC1 和最后一个 FC 之间确实用了 ReLU。

FC2

C++

```
FCLayer(120, classCount)
```

意思是

$120 \rightarrow \text{classCount}$

如果你用的是 GTX660 这种标志级显卡，通常 `classCount = 43`，那么就是

$120 \rightarrow 43$

那么

$120 \times 43 + 43 = 5203$

Softmax

`classCount` 个分数 $\rightarrow$ `classCount` 个概率

这里

`classCount = 43`

43 个 logits → 43 个概率

LeNet 总结构

```
3 × 32 × 32
↓ Conv 5×5, 3→6, stride=1, padding=0
6 × 28 × 28
↓ ReLU
6 × 28 × 28
↓ MaxPool 2×2, stride=2
6 × 14 × 14
↓ Conv 5×5, 6→16, stride=1, padding=0
16 × 10 × 10
↓ ReLU
16 × 10 × 10
↓ MaxPool 2×2, stride=2
16 × 5 × 5
↓ Flatten
400
↓ FC 400→120
120
↓ ReLU
120
↓ FC 120→classCount
classCount
↓ Softmax
classCount 个概率
```

classCount=43 字母数量大约是

56195

三、Enhanced 架构参数

这里 Enhanced 是这一层

C++

```
ConvLayer(3, 32, 5, 1, 2)
ReLU
MaxPoolingLayer(2, 2)

ConvLayer(32, 64, 3, 1, 1)
ReLU
```

```
MaxPoolingLayer(2, 2)

ConvLayer(64, 128, 3, 1, 1)
ReLU
MaxPoolingLayer(2, 2)

Flatten

FCLayer(128 * 4 * 4, 512)
ReLU
DropoutLayer(0.5)

FCLayer(512, 256)
ReLU
DropoutLayer(0.5)

FCLayer(256, classCount)
Softmax
```

Enhanced LeNet 网络结构

Enhanced 尺寸变化

input

3 × 32 × 32

Conv1

C++

```
ConvLayer(3, 32, 5, 1, 2)
```

参数

输入通道数: 3
输出通道数: 32
卷积核大小: 5×5
stride: 1
padding: 2

卷积核大小: **5×5** 卷积核: **padding=2** 卷积尺寸保持不变

output

$32 \times 32 \times 32$

计算

$(32 + 2 \times 2 - 5) / 1 + 1 = 32$

卷积核大小

$3 \times 5 \times 5$

计算

$32 \times 3 \times 5 \times 5 + 32 = 2432$

ReLU1

$32 \times 32 \times 32 \rightarrow 32 \times 32 \times 32$

Pool1

C++

```
MaxPoolingLayer(2, 2)
```

输出

$32 \times 16 \times 16$

Conv2

C++

```
ConvLayer(32, 64, 3, 1, 1)
```

计算

输入通道数: 32

输出通道数: 64

卷积核大小: 3×3

stride: 1

padding: 1

输入: 3×3 padding=1 空间尺寸保持不变

输出:

$64 \times 16 \times 16$

每个卷积核的计算:

$32 \times 3 \times 3$

总计:

$64 \times 32 \times 3 \times 3 + 64 = 18496$

ReLU2

$64 \times 16 \times 16 \rightarrow 64 \times 16 \times 16$

Pool2

$64 \times 16 \times 16 \rightarrow 64 \times 8 \times 8$

Conv3

C++

`ConvLayer(64, 128, 3, 1, 1)`

输入是:

输入通道数: 64

输出通道数: 128

卷积核大小: 3×3

```
stride: 1  
padding: 1
```

ReLU

$$128 \times 8 \times 8$$

Exp (Euler's constant)

$$64 \times 3 \times 3$$

FLOPs

$$128 \times 64 \times 3 \times 3 + 128 = 73856$$

ReLU3

$$128 \times 8 \times 8 \rightarrow 128 \times 8 \times 8$$

Pool3

$$128 \times 8 \times 8 \rightarrow 128 \times 4 \times 4$$

Flatten

$$128 \times 4 \times 4 \rightarrow 2048$$

FC1

$$128 \times 4 \times 4 = 2048$$

FC1

C++

```
FCLayer(128 * 4 * 4, 512)
```

计算量

2048 → 512

计算量

$2048 \times 512 + 512 = 1049088$

ReLU4

计算量 ReLU

512 → 512

Dropout1

C++

```
DropoutLayer(0.5F)
```

训练时随机丢弃 50% 的神经元输出

训练时随机丢弃

512 → 512

FC2

C++

```
FCLayer(512, 256)
```

计算量

512 → 256

$$512 \times 256 + 256 = 131328$$

ReLU5

这里也有 ReLU

$$256 \rightarrow 256$$

Dropout2

C++

```
DropoutLayer(0.5F)
```

这里也有 ReLU

$$256 \rightarrow 256$$

FC3

C++

```
FCLayer(256, classCount)
```

这里，FCLayer 并不是只有 FC1 和 FC2 两个全连接层，而是有 三个全连接层

FC1: $2048 \rightarrow 512$

FC2: $512 \rightarrow 256$

FC3: $256 \rightarrow \text{classCount}$

这里 `classCount=43`

$$256 \rightarrow 43$$

$$256 \times 43 + 43 = 11051$$

Softmax

classCount 个分数 $\rightarrow$ classCount 个概率

和号是

43 个 logits $\rightarrow$ 43 个概率

Enhanced 总结构

```
3 × 32 × 32
↓ Conv 5×5, 3→32, stride=1, padding=2
32 × 32 × 32
↓ ReLU
32 × 32 × 32
↓ MaxPool 2×2, stride=2
32 × 16 × 16
↓ Conv 3×3, 32→64, stride=1, padding=1
64 × 16 × 16
↓ ReLU
64 × 16 × 16
↓ MaxPool 2×2, stride=2
64 × 8 × 8
↓ Conv 3×3, 64→128, stride=1, padding=1
128 × 8 × 8
↓ ReLU
128 × 8 × 8
↓ MaxPool 2×2, stride=2
128 × 4 × 4
↓ Flatten
2048
↓ FC 2048→512
512
↓ ReLU
512
↓ Dropout 0.5
512
↓ FC 512→256
256
↓ ReLU
256
↓ Dropout 0.5
256
↓ FC 256→classCount
classCount
```

↓ Softmax
classCount 个概率

问题 classCount=43 但最初设定的是

1286251

也就是有 128.6 万参数。

四、你问：FC1 和 FC2 之间用的是 ReLU 吗？

回答：是的。

但是分类问题

如果是 LeNet

LeNet 里全连接部分是

```
Flatten
↓
FC 400→120
↓
ReLU
↓
FC 120→classCount
↓
Softmax
```

问题

FC1 和 FC2 之间有 ReLU

回答 Dropout

如果是 Enhanced

Enhanced 里全连接部分是

```
Flatten
↓
FC 2048→512
↓
```

ReLU
↓
Dropout 0.5
↓
FC 512→256
↓
ReLU
↓
Dropout 0.5
↓
FC 256→classCount
↓
Softmax

所以我在前边写

FC1 和 FC2 之间：
ReLU + Dropout

FC2 和 FC3 之间：
ReLU + Dropout

最后 FC3 和 Softmax 之间：
没有 ReLU

最后一层不能再放 ReLU，因为最后要保留每个类别的概率分布，也就是 logit，然后交给 Softmax 做概率归一化。

五、两套结构对比

项目	LeNet	Enhanced
输入	3×32×32	3×32×32
卷积层数	2	2
卷积核数	3→6→16	3→32→64→128
卷积核大小	5×5 5×5	5×5 3×3 3×3
padding	0 0	2 1 1
stride	全部 stride=1	全部 stride=1
池化核大小	2×2, stride=2	2×2, stride=2

项目	LeNet	Enhanced
Flatten 后	400	2048
全连接	400→120→classCount	2048→512→256→classCount
Dropout	无	有 0.5
训练 43 集需要时间	5.6 万	128.6 万

你现在需要确认的是：你的主程序里到底创建的是

C++

CNNArchitecture::LeNet

还是

C++

CNNArchitecture::Enhanced

从代码 `CNN.cpp` 不难看出，如果走 Enhanced，才是现在更偏正式训练用的结构。LeNet 更偏向实验的验证和半信半疑的尝试。

dropout的丢失是怎么实现的？是把随机一半的参数置零还是变成随机数

已思考几秒

Dropout 不是把参数置零，也不是把参数变成随机数

它做的是

在训练时，把某一层的所有输出值 / 激活值随机置零。

也就是说，丢失的是这一轮 for 循环里的那些神经元输出，不是权重参数本身。

比如FC1输出了6个元素：

```
[2.0, 0.5, 1.2, 3.0, 0.8, 4.0]
```

如果 `Dropout(0.5)` 表示它的随机丢掉一半。

可能生成一个如此 mask

```
[1, 0, 1, 0, 1, 0]
```

然后输出变为

```
[2.0, 0, 1.2, 0, 0.8, 0]
```

但此时使用的是 `inverted dropout`，所以保留下来的值还会放大：

```
scale = 1 / (1 - rate)
```

如果 `rate = 0.5`

```
scale = 1 / 0.5 = 2
```

所以最后输出是

```
[4.0, 0, 2.4, 0, 1.6, 0]
```

为什么需要放大？

因为训练时随机丢了一半神经元，如果不放大，整个输出平均值会变小。

用 `inverted dropout` 后

训练时：

随机丢掉一部分输出，保留下来的放大

推理时：

不丢弃，直接原样输出

在神经网络中就叫做 dropout 层结构

回到你的代码，`Enhance` 网络里是：

C++

```
FCLayer(128 * 4 * 4, 512)
ReLU
DropoutLayer(0.5)

FCLayer(512, 256)
ReLU
DropoutLayer(0.5)

FCLayer(256, classCount)
Softmax
```

问题是，Dropout 发生在

FC1 → ReLU 之后
FC2 → ReLU 之后

我们想要的是

ReLU 输出后的激活值

但是

FC1 的权重
FC2 的权重
卷积核参数
bias

这些参数是共享的，只是这一次训练时，部分神经元输出暂时不参与后续计算

反向传播时也一样

如果是在计算 forward 时把 Dropout 丢掉了

输出 = 0

那么 backward 时这个位置的梯度也会被置零

gradient = 0

因此

这一次训练中，这个被丢掉的神经元不更新相关贡献

但下一张图片，下一个 batch，它可能又被保留下来。

所以Dropout的核心是：

训练时：

随机屏蔽一部分激活值

推理时：

不屏蔽，全部使用

参数：

不会被随机置零

不会变成随机数

一句话：

Dropout 是临时随机关闭一部分神经元输出，而不是破坏或随机化模型参数。